

Mobile transport layer

Introduction

- Most applications rely on a transport layer
 - TCP (transmission control protocol) or
 - UDP (user datagram protocol)
- Two functions of the transport layer in the internet:
 - checksumming over user data
 - multiplexing or demultiplexing of data from/to applications.

Introduction

- While the network layer only addresses a host,
 - ports in UDP or TCP allow dedicated applications to be addressed.
- The connectionless UDP does not offer much more than this addressing
- The following focuses on TCP
 - UDP is connectionless and does not give certain guarantees about reliable data delivery
 - TCP is much more complex and, needs special mechanisms to be useful in mobile environments.
- Mobility support in IP (such as mobile IP) is already enough for UDP to work.

Introduction

- Within a connection TCP can give certain guarantees
 - such as in-order delivery or
 - reliable data transmission using retransmission techniques.
- TCP has built-in mechanisms to behave in a ‘network friendly’ manner.
 - If, for example, TCP encounters packet loss, it assumes network internal congestion and slows down the transmission rate.
 - This is one of the main reasons to stay with protocols like TCP.

Introduction

- One key requirement for new developments in the internet is 'TCP friendliness'
- UDP does not behave in a network friendly manner,
 - i.e., does not pull back in case of congestion and continues to send packets into an already congested network.

Introduction

- The mechanisms within TCP that play an important role when using TCP for mobility.
 - The main problem with many mechanisms is that they have been designed for situations that are completely different from those in mobile networks.

Traditional TCP

Mechanisms that influence the performance of TCP:

- Congestion Control
- Slow start
- Fast retransmit/fast recovery

Congestion Control

- TCP has been designed for fixed networks with fixed end-systems.
- Data transmission takes place without introducing transmission errors.
 - The probable reason for a packet loss in a fixed network is a state of congestion at a node.
 - »A router drops packets.
- Retransmitting the missing packet and continuing at full sending rate would now be unwise, as this might only increase the congestion.

Congestion Control

- To mitigate congestion, TCP slows down the transmission rate dramatically.
- All other TCP connections experiencing the same congestion do exactly the same so the congestion is resolved.
 - This cooperation of TCP connections is one of the main reasons for its survival.

- **Congestion Control**

- Using UDP is not a solution
 - the throughput is higher just at the beginning.
 - everyone uses UDP, this advantage disappears.
 - data transmission quality is unpredictable.
- Under heavy load, TCP guarantees at least sharing of the bandwidth.

- **Slow start**

- Exponential growth of the congestion window in the slow start mechanism
- As soon as the congestion window reaches the congestion threshold, further increase of the transmission rate is only linear by adding 1 to the congestion window each time the acknowledgements come back.

- **Slow start**

- Linear increase continues until a time-out at the sender occurs and the sender sets the congestion threshold to half of the current congestion window.
 - The congestion window itself is set to one segment and the sender starts sending a single segment.
 - The exponential growth starts once more up to the new congestion threshold, then the window grows in linear fashion.

• **Fast retransmit/fast recovery**

- When a sender receiving continuous ACKs for the same packet. It informs two things to the sender.
 - The receiver got all packets up to the ACKed packet in sequence. Receiving ACKs from a receiver also shows that the receiver continuously receives something from the sender.
 - The gap in the packet stream is due to a transmission error. The sender can now retransmit the missing packet(s) before the timer expires. This behavior is called fast retransmit.

• **Fast retransmit/fast recovery**

- After receipt of acknowledgements, the sender performs a fast recovery from the packet loss.
 - This can improve the efficiency of TCP dramatically.
- The reason for activating slow start is a time-out due to a missing acknowledgement.
 - TCP using fast retransmit/fast recovery interprets this congestion in the network and activates the slow start mechanism.

Implications on mobility

- Slow Start drastically decreases the efficiency of TCP if used together with mobile receivers or senders.
 - The reason for this is the use of slow start under the wrong assumptions.
- From a missing ACK, TCP concludes a congestion situation.
 - this may also happen in networks with mobile and wireless end-systems
 - it is not the main reason for packet loss.

Implications on mobility

- Error rates on wireless links are orders of magnitude higher compared to fixed fiber or copper links.
- Packet loss is much more common and cannot always be compensated for by layer 2 retransmissions (ARQ) or error correction (FEC).
- Trying to retransmit on layer 2 could, for example, trigger TCP retransmission if it takes too long.
 - Layer 2 now faces the problem of transmitting the same packet twice over a bad link.

Implications on mobility

- Mobility itself can cause packet loss.
 - situations where a soft handover from one access point to another is not possible for a mobile end system.
 - For example, when using **mobile IP**, there could still be some packets in transit to the old foreign agent while the mobile node moves to the new foreign agent.
 - The old foreign agent **may not** forward to new agent and buffer the packets if **disconnection** of the mobile node takes **too long**.
 - This packet loss has **nothing to do with wireless access** but is caused by the problems of rerouting traffic.

Implications on mobility

- The TCP mechanism detecting missing ACKs via time-outs and concluding packet loss due to congestion
 - cannot distinguish between the different causes.
 - This is a fundamental design problem in TCP
 - An error control mechanism is misused for congestion control.
- In both cases packets are lost (either due to invalid checksums or to dropping in routers). However, the reasons are completely different.
 - TCP cannot distinguish between these two different reasons.

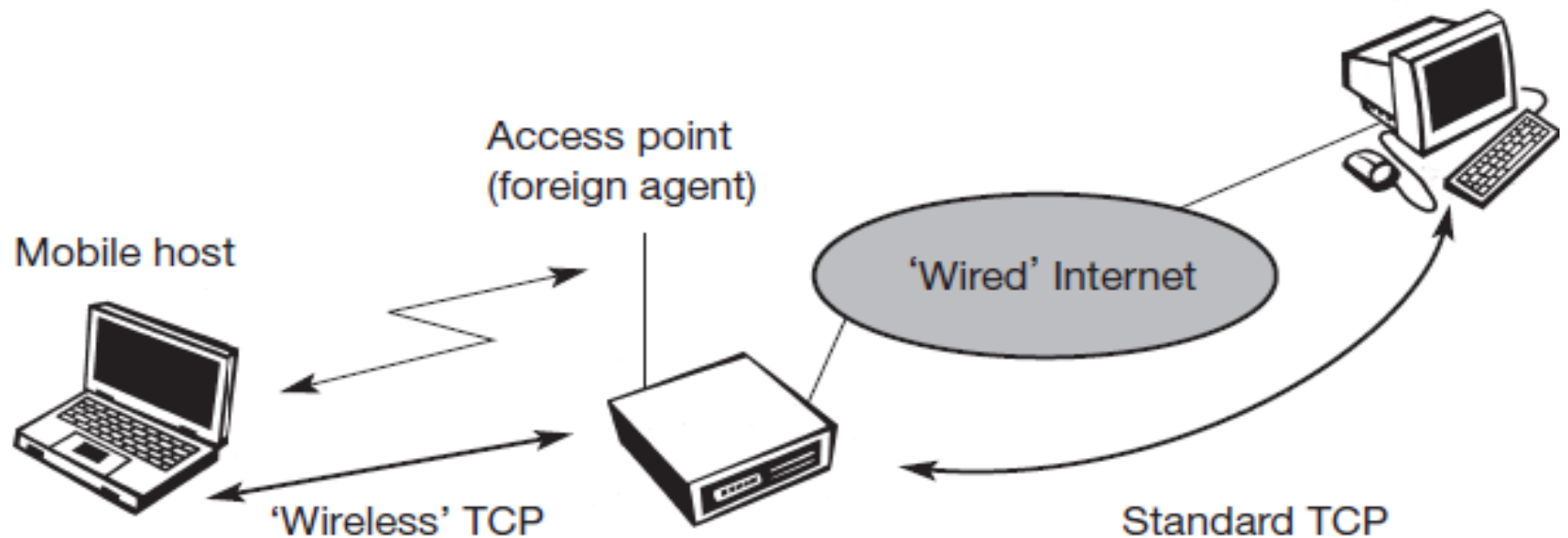
Implications on mobility

- Standard TCP reacts with slow start if ACKs are missing, which does not help in the case of transmission errors over wireless links and which does not really help during handover.
 - This behavior results in a severe performance degradation of an unchanged TCP if used together with wireless links or mobile nodes.

Implications on mobility

- One cannot change TCP completely just to support mobile users or wireless links.
- The same arguments that were given to keep IP unchanged also apply to TCP.
 - The installed base of computers using TCP is too large to be changed and, more important, mechanisms such as slow start keep the internet operable.
- Every enhancement to TCP, therefore, has to remain compatible with the standard TCP and must not jeopardize the cautious behavior of TCP in case of congestion.

Classical TCP improvements



- Indirect TCP
 - Two competing insights led to the development of indirect TCP (I-TCP):
 - TCP performs poorly together with wireless links
 - TCP within the fixed network cannot be changed
 - I-TCP segments a TCP connection into a fixed part and a wireless part.

Indirect TCP

- Standard TCP is used between the fixed computer and the access point.
 - The access point now terminates the standard TCP connection.
- Between the access point and the mobile host,
 - a special TCP, adapted to wireless links, is used.
 - changing TCP for the wireless link is not a requirement.
 - Even an unchanged TCP can benefit from the much shorter round trip time, starting retransmission much faster.

Indirect TCP

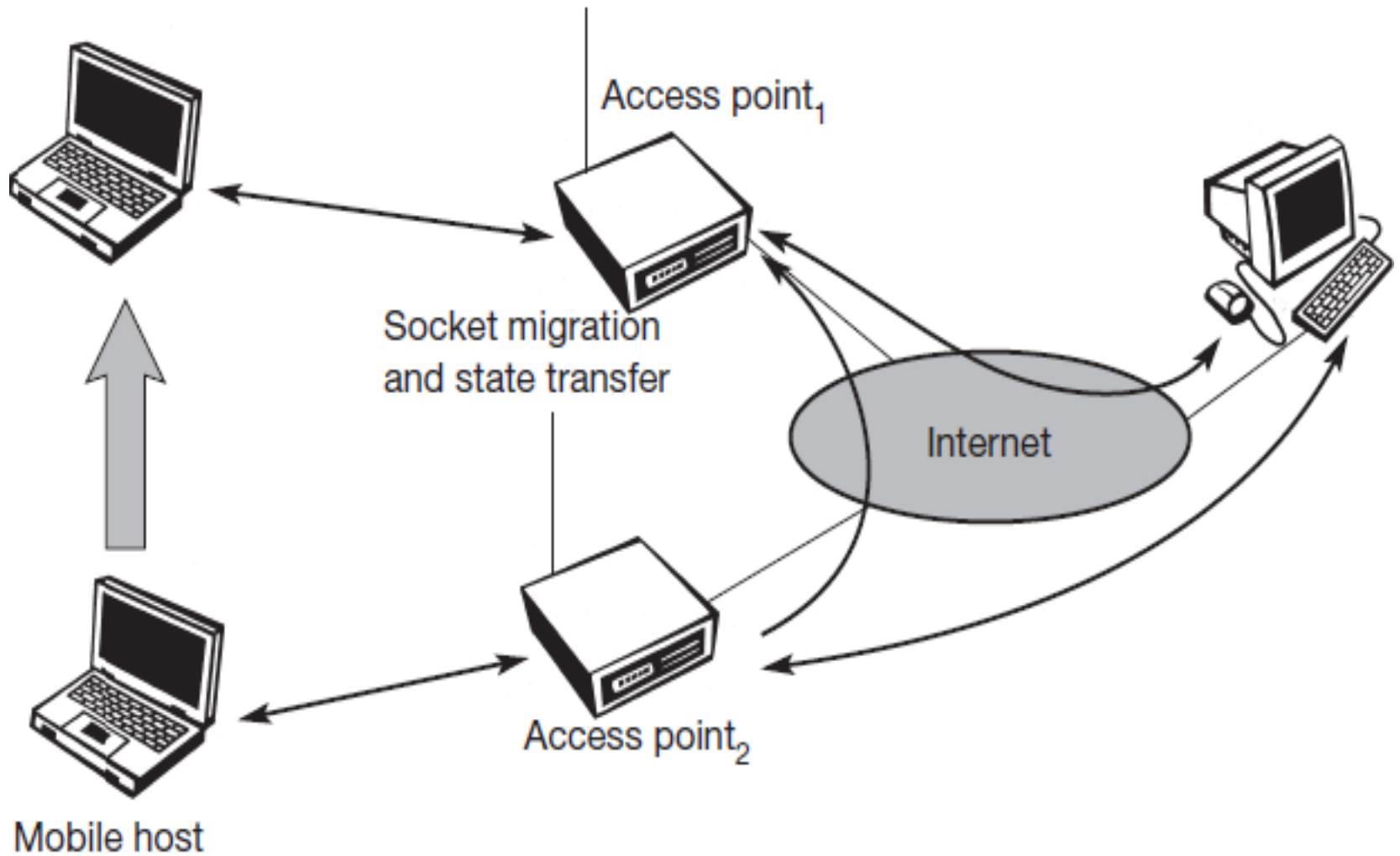
- A good place for segmenting the connection between MN and CN is **at the foreign agent** of mobile IP.
- The CN in the fixed network does not notice the wireless link or the segmentation of the connection.
- The foreign agent acts as a proxy and relays all data in both directions.

Indirect TCP

- If the CN sends a packet, the foreign agent acknowledges this packet and tries to forward the packet to the MN.
- If the MN receives the packet, it acknowledges the packet.
 - this ACK is only used by the foreign agent.
- If a packet is lost on the wireless link due to a transmission error, the CN would not notice this.
 - In this case, the foreign agent tries to retransmit this packet locally to maintain reliable data transport.

Indirect TCP

- Similarly, if the MN sends a packet, the foreign agent acknowledges this packet and tries to forward it to the CN.
- If the packet is lost on the wireless link, the MNs notice this much faster due to the lower round trip time and can directly retransmit the packet.
- Packet loss in the wired network is now handled by the foreign agent.



Socket and state migration after handover of a mobile host

Indirect TCP

- I-TCP requires several actions as soon as a handover takes place.
 - The packets have to be redirected using, e.g., mobile IP.
 - The access point acts as a proxy buffering packets for retransmission.
 - the old proxy must forward buffered data to the new proxy because it has already acknowledged the data.
 - Besides buffer content, the sockets of the proxy, too, must migrate to the new foreign agent located in the access point.
 - The socket reflects the current state of the TCP connection, i.e., sequence number, addresses, ports etc.
 - No new connection may be established for the MN, and the CN must not see any changes in connection state.

Advantages

- I-TCP does not require any changes in the TCP protocol as used by the hosts in the fixed network, or other hosts in a wireless network that do not use this optimization.
 - All current optimizations for TCP still work between the foreign agent and the correspondent host.
- Due to the strict partitioning into two connections, transmission errors on the wireless link, i.e., lost packets, cannot propagate into the fixed network.
 - Now only packets in sequence, without gaps leave the foreign agent.

Advantages

- always dangerous to introduce new mechanisms into a huge network, but with I-TCP only between MN and the FA.
 - Different solutions can be tested or used at the same time without jeopardizing the stability of the internet.
 - optimizing of these new mechanisms is quite simple because they only cover one single hop.
- Due to the short delay between the MN and FA, an optimized TCP could use precise time-outs to guarantee retransmission as fast as possible.
 - Even standard TCP could benefit from the short round trip time, so recovering faster from packet loss.

Advantages

- Partitioning into two connections also allows the use of a different transport layer protocol between the FA and the MN or the use of compressed headers etc.
 - The foreign agent can now act as a gateway to translate between the different protocols.

Disadvantage

- The loss of the end-to-end semantics of TCP might cause problems if the foreign agent partitioning the TCP connection crashes.
- If a sender receives an acknowledgement, it assumes that the receiver got the packet.
 - Receiving an Ack now only means that the FA received the packet. It may also crash applications running on the CN assuming reliable end-to-end delivery.

Disadvantage

- In practical use, increased handover latency may be much more problematic.
 - FA buffers the packets sent by CN.
 - FA removes the packet after receiving Acks from the MN.
 - Due to handover by MN, it takes a while before the old FA can forward the buffered data to the new FA. During this time more packets may arrive. All these packets have to be forwarded to the new foreign agent first, before it can start forwarding the new packets redirected to it.
- The FA must be a trusted entity because the TCP connections end at this point. If users apply end-to-end encryption, the foreign agent has to be integrated into all security mechanisms.